

Feb 08, 2025 15:15

ControlDB.py

Page 1/12

```
1  # this controls the database interactions in pyside6
2  import sys
3  import os
4  import platform
5  from loguru import logger
6
7
8  import config_mycal
9  from Recipe import Ui_MainWindow
10
11
12  from PySide6.QtCore import (QSize, Qt ,QRect,
13  QCoreApplication,Slot,Signal,QAbstractListModel,QMetaObject)
14  from PySide6.QtGui import QAction,QDoubleValidator,QFont
15  from PySide6.QtWidgets import QWidget
16  from PySide6.QtUiTools import QUiLoader
17
18  from PySide6.QtSql import QSqlDatabase , QSql,QSqlTableModel,QSqlQueryModel,QSqlQuery
19
20  from PySide6.QtWidgets import (
21  QApplication,
22      QCheckBox,
23      QComboBox,
24      QDateEdit,
25      QDateTimeEdit,
26      QDial,
27      QDoubleSpinBox,
28      QFileDialog,
29      QFontComboBox,
30      QFormLayout,
31      QHBoxLayout,
32      QLabel,
33     QLCDNumber,
34      QLineEdit,
35      QListView,
36      QMainWindow,
37      QProgressBar,
38      QPushButton,
39      QRadioButton,
40      QSlider,
41      QSpinBox,
42      QStatusBar,
43      QTimeEdit,
44      QTableView,
45      QMenuBar,
46      QVBoxLayout,
47      QAbstractItemView,
48      QGridLayout,
49      QTextEdit
50  )
51
52
53  class NewWindow(QMainWindow):
54      def __init__(self,Title = None):
55          super().__init__()
56          layout = QVBoxLayout ()
57
```

```

58         self.label = QLabel("Another Window")
59         if(Title == None):
60             self.setWindowTitle("MyCal")
61         else:
62             self.setWindowTitle(Title)
63
64         layout.addWidget(self.label)
65         self.setLayout(layout)
66
67
68     class RecipeModel(QAbstractListModel):
69
70         def __init__(self, recipes=None):
71             super().__init__()
72
73         #When subclassing QAbstractListModel, you must provide implementations of the rowCount() and data() functions.
74         # Well behaved models also provide a headerData() implementation.
75         # this means we will have to define the data and rowCount
76         # see also https://doc.qt.io/qt-6/qabstractlistmodel.html
77
78         self.recipes = recipes or []
79
80         def data(self, index, role):
81             if role == Qt.DisplayRole:
82                 text = self.recipes[index.row()]
83                 return text
84
85
86         def rowCount(self, index):
87             return len(self.recipes)
88
89     class MyRecipeWindow(QMainWindow, Ui_MainWindow):
90         def __init__(self):
91             super().__init__()
92             self.setupUi(self)
93
94
95     class ControlDB(QMainWindow):
96
97         def __init__(self, Title=None, db_name=None, db_user=None, db_system=None, db_pwd = None):
98             super().__init__()
99             self.db_name = db_name
100            self.db_user = db_user
101            self.db_system = db_system
102            if db_pwd == None:
103
104                temp = '/Users/klein/git/qt_exercises/config/pw.txt'
105                if os.path.exists(temp):
106                    with open(temp, 'r') as file:
107                        password = file.read().rstrip()
108                        self.db_pwd = password
109                else:
110                    self.db_pwd = db_pwd
111
112
113            self.SetupLogger()
114            #self.CreateIngredientsForm()

```

Feb 08, 2025 15:15

ControlDB.py

Page 3/12

```

115         self.ingredients_suppress_columns = ['Sugar',
116                                             'Portion_size',
117                                             'Saturated',
118                                             'Fiber',
119                                             'Salt',
120                                             'Sodium',
121                                             'Product_source'
122                                             ]
123
124         self.recipe_suppress_columns = ['Sugar',
125                                         'Portions',
126                                         'Description',
127                                         'Saturated_fat',
128                                         'Fiber',
129                                         'Salt',
130                                         'Sodium',
131                                         'Ingredients',
132                                         'Time',
133                                         'Images',
134                                         'vegetarian' ]
135
136         self.version = '0.1'
137         logger.info('***** \n\n\n')
138
139         logger.info(' This is version %s' % self.version)
140         logger.info('\n\n\n***** \n')
141
142
143
144
145     def ConnectDataBase(self):
146         ''' establish contact to database'''
147
148
149         #instantiate the connection
150         self.mycal_db = QSqlDatabase.addDatabase(self.db_system)
151         self.mycal_db.setHostName("localhost")
152         self.mycal_db.setDatabaseName(self.db_name)
153         self.mycal_db.setUserName(self.db_user)
154         self.mycal_db.setPassword(self.db_pwd)
155
156
157     #         self.mycal_db.setPassword(self.CM.pwd)
158
159     result = self.mycal_db.open()
160     if(result):
161         logger.info('connection successful')
162         # here we get the connection name
163         # will be needed when we want to close the connection
164         self.connection_name = self.mycal_db.connectionName()
165         logger.info(' database connection name %s' % self.connection_name)
166         #self.ShowTables()
167     else:
168         logger.error('connection failed, exiting')
169         sys.exit(0)
170
171

```

```

172     def ShowTables(self) :
173         '''prints all the tables in the databe'''
174         self.table_list = self.mycal_db.tables(type=QSql.Tables)
175         for k in range(0,len(self.table_list)):
176             print(self.table_list[k])
177
178         #Create list box
179         self.mybox = QComboBox()
180         #Insert at index 0 the whole list
181         self.mybox.insertItems(0,self.table_list)
182         # create independent window
183
184         #self.setCentralWidget(self.mybox)
185
186
187     return
188
189     def ViewTable(self,table,suppress_columns=[],editmode=True,columnwidth=[],Title = None, showTable = True):
190         self.table_view = QTableView()
191
192
193         self.model = QSqlTableModel(db = self.mycal_db)
194
195
196
197         self.table_view.setModel(self.model)
198         #prevent editing
199         if(not editmode):
200             self.table_view.setEditTriggers(QAbstractItemView.NoEditTriggers)
201
202         self.model.setTable(table)
203
204         # get rid of columns
205         columns_to_remove = suppress_columns
206         for cn in columns_to_remove:
207             idx = self.model.fieldIndex(cn)
208             self.model.removeColumns(idx, 1)
209
210
211         self.model.select()
212         # change column width
213         for k in range(0,len(columnwidth)):
214             self.table_view.setColumnWidth(k, columnwidth[k])
215
216         self.setMinimumSize(QSize(300, 300))
217
218         self.setCentralWidget(self.table_view)
219
220         self.SetPosition(10,10)
221         if(showTable):
222             self.show()
223         #self.setCentralWidget(table)
224
225     def SetupLogger(self) :
226
227
228         logger.remove(0)

```

Feb 08, 2025 15:15

ControlDB.py

Page 5/12

```

229     #now we add color to the terminal output
230     logger.add(sys.stdout,
231                 colorize = True, format="<green>{time}</green> {function} {line} {level} <level>{message}</level>" ,
232                 level = "INFO")
233
234
235
236     fmt = "{time} - {name} - {function} - {line} - {level} - {message}"
237     logger.add('info.log', format = fmt , level = 'INFO', rotation="1 day")
238
239
240     # set the colors of the different levels
241     logger.level("INFO", color = '<black>')
242     logger.level("WARNING", color = '<green>')
243     logger.level("ERROR", color = '<red>')
244     logger.level("DEBUG", color = '<blue>')
245
246     return
247
248 def SetPosition(self, pos_x, pos_y):
249     self.move(pos_x, pos_y)
250
251 def CreateIngredientsForm1(self):
252     """uses the form layout"""
253
254     #always use self so window will be persistent
255     self.Inform = NewWindow(Title = "Add New Ingredient")
256     self.Inform.setStyleSheet("background-color: yellow;")
257
258     # setup geometry
259     mysize=[400,300]
260     myposit = [800,100]
261
262     self.SizeWindow(self.Inform, myposit, mysize)
263
264     # Create widgets
265     self.name_label = QLineEdit()
266     self.name_label.setStyleSheet("background-color: rgb(3, 252, 227)")
267     # force valid entr
268     double_validator = QDoubleValidator(0, 500.0, 2)
269
270     self.calory_label = QLineEdit()
271     self.calory_label.setStyleSheet("background-color: white")
272     self.calory_label.setValidator(double_validator)
273
274     self.carb_label = QLineEdit()
275     self.carb_label.setStyleSheet("background-color: white")
276     self.carb_label.setValidator(double_validator)
277     self.fat_label = QLineEdit()
278     self.fat_label.setStyleSheet("background-color: white")
279     self.fat_label.setValidator(double_validator)
280     self.prot_label = QLineEdit()
281     self.prot_label.setStyleSheet("background-color: white")
282     self.prot_label.setValidator(double_validator)
283
284     SaveButton = QPushButton("Save")
285     SaveButton.setStyleSheet("background-color: white")

```

Feb 08, 2025 15:15

ControlDB.py

Page 6/12

```

286     CancelButton = QPushButton("Cancel")
287     CancelButton.setStyleSheet("background-color: red")
288
289
290     # Create layout
291     form_layout = QFormLayout()
292     form_layout.addRow("Name:", self.name_label)
293     form_layout.addRow("Calories/100g", self.calory_label)
294     form_layout.addRow("Fat", self.fat_label)
295     form_layout.addRow("Carbohydrates", self.carb_label)
296     form_layout.addRow("Protein", self.prot_label)
297     form_layout.addRow(SaveButton)
298     form_layout.addRow(CancelButton)
299
300     mylayout = QVBoxLayout()
301     mylayout.addLayout(form_layout)
302     self.setLayout(mylayout)
303
304     # Connect submit button
305     SaveButton.clicked.connect(self.SaveIngredients)
306     CancelButton.clicked.connect(lambda : self.Cancel(self.Inform))
307
308     widget = QWidget()
309     widget.setLayout(mylayout)
310
311
312
313
314
315     self.Inform.setCentralWidget(widget)
316
317     #for label in MyLabel:
318     #    widget = QWidget()
319     #    widget.setLayout(layout)
320     #self.setCentralWidget(widget)
321
322
323     # create the form
324
325     self.Inform.show()
326
327
328     def InsertRecord(self, table = None):
329         """ this inserts a record into the cosen table
330         I will use the query function to do this.
331         The record contains the info to be added
332         record=[name,energy,fat,carbohydrate,protein]
333
334         This is the same for ingredients and receipes"""
335
336     #Create table view
337     if(table == "ingredients"):
338
339         self.ViewTable(table = table,suppress_columns=self.ingredients_suppress_columns,showTable= False)
340     else:
341         self.ViewTable(table = table,suppress_columns=self.recipe_suppress_columns)
342

```

Feb 08, 2025 15:15

ControlDB.py

Page 7/12

```

343     self.model = QSqlQueryModel()
344     self.table_view.setModel(self.model)
345     #next we check we don't have an entry yet
346
347     #Form the sql INSERT statement
348     # self.record = [Ing_name,Ing_calory,Ing_carb,Ing_fat,Ing_prot]
349     #INSERT INTO products (product_no, name, price) VALUES (1, 'Cheese', 9.99);
350     #INSERT INTO table (name,energy,carbohydrate,fat,protein) VALUES (self.record[0],
351     #                                                                    self.record[1],
352     #                                                                    self.record[2],
353     #                                                                    self.record[3],
354     #                                                                    self.record[4]);
355     temp = ""+self.record[0]+" "
356     sql = 'INSERT INTO '+table+' (name,energy,carbohydrate,fat,protein) VALUES ('+temp+','+str(self.record[1])+','+str(self.record[2])+','+str(
self.record[3])+','+str(self.record[4])+');'
357     print(sql)
358
359     query = QSqlQuery(sql,db=self.mycal_db)
360
361     self.model.setQuery(query)
362     logger.info(' inserted record %s into table %s' % (self.record[0],table))
363
364
365
366     return
367
368 def MyRecipes_new(self):
369     """using the QTdesigner"""
370     self.MRD = MyRecipeWindow()
371     self.MRD.show()
372
373     # Load the model
374     self.RecipeModel = RecipeModel()
375     self.MRD.RecipeView.setModel(self.RecipeModel)
376
377     #poulate the combo box
378     self.MRD.RecipeListCombo.setObjectName(u"RecipeListCombo")
379     self.MRD.RecipeListCombo.addItem(self.GetRecipeList())
380
381
382     #here we define the actions
383     self.MRD.RecipeListCombo.currentTextChanged.connect(self.getSelectedRecipe)
384
385
386
387
388
389
390 def getSelectedText(self,s):
391     self.selected_text = s
392     logger.info("You have selected %s text" % self.selected_text)
393     return
394
395
396 def getSelectedRecipe(self,s):
397     #Clear Buffer of recipes
398

```

```

399
400     self.RecipeModel.recipes = []
401
402     self.selected_recipe = s
403     logger.info("You have selected %s recipe" % self.selected_recipe)
404
405     # now we populate the listview
406     self.FillRecipeIngredientList()
407
408     # Now fill qlist
409     self.DisplyRecipeList()
410
411     # send signal list has been updated
412     self.RecipeModel.layoutChanged.emit()
413
414
415
416     return
417
418 def FillRecipeIngredientList(self):
419     # here we get the ingredients from the Recipe database
420     # this is done with a query
421
422
423
424     sql = ' SELECT ingredients from recipes WHERE recipes.name LIKE \'\' + self.selected_recipe + \'\' ;'
425     query = QSqlQuery(sql, db=self.mycal_db)
426     model = QSqlQueryModel()
427
428     model.setQuery(query)
429     while query.next():
430
431         temp_value = ((query.value(0)))
432         # now we have to work with string
433         # the old database still had *u000a in front of the number
434         # need first to strip this
435         #check if temp contains *u000a
436         mask = 'u000a'
437         if(mask in temp_value):
438             temp_new = self.StripUnicode(temp_value, mask)
439             #temp_new = self.StripUnicode(temp_new, '*')
440
441
442
443
444         else:
445             temp_new = temp_value
446
447         delimiter = "g"
448         result = temp_new.replace('u000a', '')
449
450
451         result1 = self.split_and_keep(result, delimiter)
452         # strip last *
453         result11= result1[0]
454         result2 = result11[:len(result1[0])-1].split('*')
455         # we know that a string with underscores has a max of three spaces. Any number larger than this means words with spaces.

```


Feb 08, 2025 15:15

ControlDB.py

Page 9/12

```

456         #in other words 4 spaces means the 3rd has to be replaced with _ 5 spaces mean the 3rd and 4th
457         result3 = []
458         for a in result2:
459             space_indices = []
460             for i, char in enumerate(a):
461                 if char == ' ':
462                     space_indices.append(i)
463
464             if len(space_indices) == 4:
465                 na = a[:space_indices[2]] + '_' + a[space_indices[2] + 1:]
466
467                 result3.append(na)
468
469             elif len(space_indices) == 5:
470                 na = a[:space_indices[2]] + '_' + a[space_indices[2]+1:space_indices[3]] + '_' + a[space_indices[3] + 1:]
471                 result3.append(na)
472
473             else:
474                 result3.append(a)
475
476
477         recipe_string = result3
478
479         self.recipe_ingredients = result3
480         return
481
482     def DisplyRecipeList(self):
483         """loop through ingredient list and add"""
484         for text in self.recipe_ingredients:
485             self.RecipeModel.recipes.append(text)
486         return
487
488
489     def split_and_keep(self, text, delimiter):
490         """deals with spaces instead of underscores"""
491         result = []
492         temp = ""
493         for char in text:
494             if char == delimiter:
495                 result.append(temp + char)
496                 temp = ""
497             else:
498                 temp += char
499         if temp:
500             result.append(temp)
501         return result
502
503
504
505
506     def StripUnicode(self, mystring, mask):
507         newstring = mystring.replace(mask, '')
508         return newstring
509
510
511     def GetRecipeList(self):
512         """sql query to get all the names of the recpies

```

```

513     will be used to populate the recipe qlistbox"""
514     sql = 'SELECT name from recipes;'
515     query = QSqlQuery(sql, db=self.mycal_db)
516     model = QSqlQueryModel()
517
518     model.setQuery(query)
519     recipe_list = []
520     while query.next():
521         recipe_list.append(str(query.value(0)))
522
523     return (recipe_list)
524
525
526
527
528
529     def ModifyRecipe(self):
530         pass
531
532
533     def SizeWindow(self, window, myposit, mysize):
534         """resizes the specified window, where the parameters are two integer lists"""
535         window.setGeometry(myposit[0], myposit[1], mysize[0], mysize[1])
536         return
537
538
539     #####
540     # section for signals and slots
541
542
543     def Cancel(self, window):
544         """ closes window without anything"""
545         window.destroy()
546
547
548
549
550     def SaveIngredients(self):
551         """saving the ingredients"""
552         Ing_name = self.name_label.text()
553         Ing_calory = float(self.calory_label.text())
554         Ing_carb = float(self.carb_label.text())
555         Ing_fat = float(self.fat_label.text())
556         Ing_prot = float(self.prot_label.text())
557
558         # this is the record we got from the form input
559
560         #Check all fields are filled out
561         if(Ing_calory != "" and Ing_carb != "" and Ing_fat != "" and Ing_prot != ""):
562             self.Inform.destroy() # destrorys the window.
563             #self.Inform.close() #
564
565         #now we need to add the values to the ingredients table
566         # first load table
567         table_name = 'ingredients'
568         self.ViewTable(table_name, suppress_columns=self.ingredients_suppress_columns)
569         self.model = QSqlQueryModel()

```

Feb 08, 2025 15:15

ControlDB.py

Page 11/12

```

570     self.table_view.setModel(self.model)
571     #next we check we don't have an entry yet
572
573
574     #sql = "SELECT name from ingredients WHERE ingredients.name LIKE 'Almond%' ; "
575     sql = "SELECT name from ingredients WHERE ingredients.name LIKE '"+Ing_name+"' ; "
576     query = QSqlQuery(sql,db=self.mycal_db)
577
578     self.model.setQuery(query)
579     while query.next():
580         logger.error(' entry name already exists, try again %s' % query.value(0))
581         self.CreateIngredientsForm1()
582         #myvalue = query.value(0)
583         # Now we need to add this ingredient to the table
584
585     self.record = [Ing_name,Ing_calory,Ing_carb,Ing_fat,Ing_prot]
586     self.InsertRecord(table = table_name)
587     return
588
589
590 app = QApplication(sys.argv)
591
592 Title=None
593 db_name='recipe_ak'
594 db_user='klein'
595 db_system='QPSQL'
596 db_pwd = None
597
598 suppress_columns = ['Sugar',
599                    'Portions',
600                    'Description',
601                    'Saturated_fat',
602                    'Fiber',
603                    'Salt',
604                    'Sodium',
605                    'Ingredients',
606                    'Time',
607                    'Images',
608                    'vegetarian']
609 columnwidth = [40,200,200,200,200,200]
610
611 window = ContrlDB(Title = "ControlDB",
612                  db_name=db_name,
613                  db_user=db_user,
614                  db_system=db_system)
615 #window.SetSize(800,500)
616 #window.SetPosition(100,500)
617
618 window.setStyleSheet("background-color: white;")
619 #window.CreateCalendar()
620
621
622
623 #window.show()
624 window.ConnectDataBase()
625 window.ShowTables()
626 #window.ViewTable('Recipes',suppress_columns=suppress_columns,columnwidth=columnwidth)

```

Feb 08, 2025 15:15

ControlDB.py

Page 12/12

```
627 window.MyRecipes_new()  
628 #window.CreateIngredientsForm1()  
629  
630 # now run the app  
631 app.exec()
```